

UNIT 4 – Python File Operations

Python File Operations: Reading files, Writing files in python, Understanding read functions, read(), readline(), readlines(). Understanding write functions, write() and writelines() Manipulating file pointer using seek Programming, using file operations.

1. What is File Handling?

👉 Normally, when we run a Python program, all data is stored in **RAM (temporary memory)**.

Once the program ends, all data **disappears**.

To **save data permanently**, we use **Files** stored in the computer's **hard disk**.

File handling lets our program **read** and **write** data to these files.

Real-Life Example:

When you play a game → your **score is saved** in a file.

When you use an app → your **login info** or **user data** is saved in a file.

In Data Science → datasets (data . csv) are **read from files**.

2. Opening a File

We use the open() function to open a file.

Syntax:

```
file_object = open("filename", "mode")
```

Mode	Meaning	Use
"r"	Read	Opens an existing file for reading
"w"	Write	Creates new file / erases old data
"a"	Append	Adds data at the end
"r+"	Read + Write	File must exist
"w+"	Write + Read	Creates new file, deletes old content
"a+"	Append + Read	Adds data at end, can read too

3. Reading a File

Let's say we have a text file named "gkm.txt" with the following content:

```
Hello, my name is GKM.  
I study Data Science.  
I love Python!
```

Example 1: Reading Full File

```
f = open("gkm.txt", "r")    # open file for reading  
data = f.read()             # read full content  
print(data)  
f.close()                   # close file
```

Output:

```
Hello, my name is GKM.  
I study Data Science.  
I love Python!
```

Think: `read()` → reads **entire file at once**.

Don't use it for very large files (may slow your program).

Example 2: Reading Limited Characters

```
f = open("gkm.txt", "r")  
data = f.read(10)  # reads first 10 characters  
print(data)  
f.close()
```

Output:

```
Hello, my
```

👉 `read(n)` → reads only first *n* characters.

Example 3: Reading One Line at a Time

```
f = open("gkm.txt", "r")  
print(f.readline())  # Reads 1st line  
print(f.readline())  # Reads 2nd line  
f.close()
```

Output:

Hello, my name is GKM.
I study Data Science.

💡 *Tip:* Use `readline()` when reading files line by line (like logs or CSVs).

Example 4: Reading All Lines into a List

```
f = open("gkm.txt", "r")
lines = f.readlines()
print(lines)
f.close()
```

Output:

```
['Hello, my name is GKM.\n', 'I study Data Science.\n', 'I love Python!']
```

`readlines()` stores each line as one **element in a list**.

Useful when you want to process file line by line later.

4. Writing to a File

We use "w" (write) or "a" (append) mode.

Example 1: Writing New File

```
f = open("notes.txt", "w")
f.write("This is my first file.\n")
f.write("I am learning Python File Handling.")
f.close()
```

- ✓ Creates new file → `notes.txt`
- ✓ Overwrites old content if file already exists.

Output inside notes.txt:

This is my first file.
I am learning Python File Handling.

Example 2: Appending Data

```
f = open("notes.txt", "a")
f.write("\nThis line is added later.")
f.close()
```

Now file content becomes:

```
This is my first file.
I am learning Python File Handling.
This line is added later.
```

Tip:

"w" → overwrite
"a" → add new data

5. Closing a File

Always close the file after reading or writing.

```
f.close()
```

⚠ If you forget to close the file:

- Data may not save properly.
- System memory might stay busy.

6. The Smart Way – Using with

Python gives a better way to handle files using with.

It **automatically closes** the file for you.

```
with open("notes.txt", "r") as f:
    data = f.read()
    print(data)
```

✅ No need to call `f.close()`

✅ Safer and cleaner code

7. Complete Example

```
# Writing data into file
with open("student.txt", "w") as f:
```

```
f.write("Name: GKM\n")
f.write("Course: B.Tech Data Science\n")
f.write("Year: 2nd Year\n")

# Reading data from file
with open("student.txt", "r") as f:
    print("File Content:\n")
    print(f.read())
```

Output:

```
File Content:
Name: GKM
Course: B.Tech Data Science
Year: 2nd Year
```

8. Connection to Data Science

In Data Science, we often **read large datasets** stored in files.

Example:

```
with open("data.csv", "r") as f:
    for line in f:
        print(line.strip().split(","))
```

💡 This helps to understand how libraries like **Pandas** work internally when you use `pd.read_csv()`.

✅ Easy Formula to Remember:

Task	Mode	Function
Read entire file	"r"	<code>read()</code>
Read line by line	"r"	<code>readline()</code>
Read as list	"r"	<code>readlines()</code>
Write (erase old)	"w"	<code>write()</code>
Add new data	"a"	<code>write()</code>

1. Writing Data to Files

When we open a file in **write mode ("w")** or **append mode ("a")**, we can send (write) data from our program into the file.

1.1 write() Function

👉 Used to write a **single string** into a file.

Syntax:

```
file.write(string)
```

Example 1:

```
f = open("notes.txt", "w")    # open in write mode
f.write("Hello, my name is GKM.\n")
f.write("I am learning Python File Handling.\n")
f.close()
```

- ✅ Creates a new file **notes.txt**
- ✅ Overwrites old data if file already exists

Inside notes.txt:

```
Hello, my name is GKM.
I am learning Python File Handling.
```

Note: You need to add \n (newline) manually if you want each sentence on a new line.

Example 2: Using a (append mode)

```
f = open("notes.txt", "a")    # open in append mode
f.write("This line will be added at the end.\n")
f.close()
```

Result in file:

```
Hello, my name is GKM.
I am learning Python File Handling.
This line will be added at the end.
```

- ✔ "a" keeps old data and adds new data at the end.

2. writelines() Function

- 👉 Used to write **multiple lines at once** using a **list of strings**.

Syntax:

```
file.writelines(list_of_strings)
```

Example 1:

```
f = open("students.txt", "w")
lines = ["Gautam\n", "Neeraj\n", "Rohan\n", "Ananya\n"]
f.writelines(lines)
f.close()
```

- ✔ Each list element is written as one line.

Inside students.txt:

```
Gautam
Neeraj
Rohan
Ananya
```

- 💡 *Important:* writelines() **does not automatically add \n**, so we must include it inside each string if we want new lines.

Example 2:

```
names = ["Aman", "Sita", "Ravi"]
with open("class.txt", "w") as f:
    f.writelines(name + "\n" for name in names)
```

- ✔ Better way — adds new lines automatically using loop and generator.

3. File Pointer and seek() Function

Whenever you open a file, Python uses an **invisible pointer (cursor)** to keep track of **where you are** in the file.

Think of it like a **cursor in MS Word** that moves as you type or read text.

🔗 3.1 The tell() Function

👉 Shows the **current position** of the file pointer.

```
f = open("demo.txt", "r")
print(f.tell())          # Position before reading
data = f.read(5)
print(data)
print(f.tell())          # Position after reading 5 chars
f.close()
```

Example Output:

```
0
Hello
5
```

💡 File pointer moves forward as you read/write data.

3.2 The seek() Function

👉 Used to **move the pointer** to a specific position in the file.

Syntax:

```
file.seek(offset, from_what)
```

Parameter	Meaning
offset	Number of bytes to move
from_what	Starting point (0 = start, 1 = current, 2 = end)

Usually, we use:

- seek(0) → Move to beginning
- seek(10) → Move to 10th byte

Example 1: Reset Pointer

```
f = open("demo.txt", "r")
data1 = f.read(10)
print("Data1:", data1)

f.seek(0)          # move pointer to start
data2 = f.read(5)
```



```
print("Data2:", data2)
f.close()
```

Output:

Data1: Hello GKM

Data2: Hello

💡 The first read moved pointer forward;
seek(0) brought it **back to start**.

Example 2: Reading From Middle

```
If file has "Data Science is fun!"
f = open("data.txt", "r")
f.seek(5)
print(f.read())
f.close()
```

Output:

Science is fun!

Pointer skipped the first 5 characters.

4. Programming Using File Operations

Let's see how all functions (read, write, seek) can work together.

Example 1: Copy File Content

```
# Copy data from one file to another
with open("source.txt", "r") as src:
    with open("copy.txt", "w") as dst:
        for line in src:
            dst.write(line)
```

✅ Reads data line-by-line and writes into another file.

👉 Used often in backup or data processing tasks.

Example 2: Count Lines, Words, and Characters

```
with open("data.txt", "r") as f:
    text = f.read()
```

```
lines = text.split("\n")
words = text.split()
print("Lines:", len(lines))
print("Words:", len(words))
print("Characters:", len(text))
```

Output:

Lines: 3
Words: 10
Characters: 56

Real-life use: reading text files, counting log entries, etc.

Example 3: Read Specific Line Twice Using seek()

```
with open("demo.txt", "r") as f:
    print("First 5 chars:", f.read(5))
    f.seek(0) # Go back to start
    print("Again first 5 chars:", f.read(5))
```

Output:

First 5 chars: Hello
Again first 5 chars: Hello

Summary (Short Notes)

Function	Purpose	Example
write()	Write single string	f.write("Hello\n")
writelines()	Write list of strings	f.writelines(list)
tell()	Show current position	f.tell()
seek(pos)	Move pointer	f.seek(0)

UNIT 4 – Python File Operations (Most Important Q&A)

Q1. What is file handling in Python? Why is it needed?

File handling means performing operations like **creating**, **reading**, **writing**, and **updating** files through Python programs.

◇ Need for File Handling:

- Variables store data **temporarily (in RAM)**.
- Files store data **permanently (on disk)**.
- Used to store user data, logs, or datasets for analysis.
-

🔗 Example:

```
f = open("data.txt", "w")
f.write("Hello, File Handling!")
f.close()
```

Q2. Explain different file modes in Python.

Mode	Meaning	Description
"r"	Read	Opens file for reading (error if not found).
"w"	Write	Opens file for writing (creates or overwrites).
"a"	Append	Opens for writing and adds data at end.
"r+"	Read + Write	Reads and writes both (file must exist).
"w+"	Write + Read	Creates new file if not exist; overwrites existing.
"a+"	Append + Read	Reads and appends new data at end.

Q3. Explain the difference between read(), readline(), and readlines() with examples.

Function	Description	Example
read()	Reads the entire file as one string.	f.read()

<code>readline()</code>	Reads one line at a time .	<code>f.readline()</code>
<code>readlines()</code>	Reads all lines into a list .	<code>f.readlines()</code>

Example:

If file `demo.txt` contains:

```
A
B
C
f = open("demo.txt", "r")
print(f.read())          # A\nB\nC
f.seek(0)
print(f.readline())      # A
f.seek(0)
print(f.readlines())     # ['A\n', 'B\n', 'C\n']
f.close()
```

Q4. Explain `write()` and `writelines()` with examples.

◇ `write()`

Writes a **single string** into a file.

```
f = open("notes.txt", "w")
f.write("Hello GKM!\n")
f.write("Python File Handling.")
f.close()
```

Output in file:

```
Hello GKM!
Python File Handling.
```

◇ `writelines()`

Writes **multiple lines** at once (expects a list of strings).

```
f = open("data.txt", "w")
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
f.writelines(lines)
f.close()
```

Output in file:

Line 1
Line 2
Line 3

Q5. What is a file pointer? How can we use seek() and tell() functions?

- File pointer = The **position** in the file where reading/writing happens.
- Python automatically moves this pointer as we read/write.

◇ Functions:

Function	Description
seek(offset, from)	Moves file pointer to a specific position.
tell()	Returns current file pointer position.

Example:

```
f = open("example.txt", "r")
print(f.read(5))      # Reads 5 characters
print(f.tell())       # Shows current position
f.seek(0)             # Moves pointer back to start
print(f.read(5))      # Reads again from beginning
f.close()
```

Q6. Write a Python program to read a file and count the number of lines.

```
count = 0
with open("sample.txt", "r") as f:
    for line in f:
        count += 1
print("Total Lines:", count)
```

Output:

Total Lines: 5

Q7. Write a Python program to copy content from one file to another.

```
with open("source.txt", "r") as f1:
    data = f1.read()
with open("destination.txt", "w") as f2:
    f2.write(data)
print("File copied successfully.")
```

Q8. Write a Python program to append new content to an existing file.

```
with open("notes.txt", "a") as f:
    f.write("\nThis line was added later.")
print("Data appended successfully.")
```

Q9. Write a Python program to count words in a file.

```
with open("story.txt", "r") as f:
    text = f.read()
words = text.split()
print("Total Words:", len(words))
```

Output:

Total Words: 35

Q10. What is the difference between text file and binary file?

Type	Description	Example
Text File	Stores data as plain text. Human-readable.	.txt, .csv

Binary File	Stores data in binary (0s and 1s). Not human-readable.	.jpg, .dat, .exe
--------------------	--	------------------

Example to open binary file:

```
f = open("image.jpg", "rb")
data = f.read()
f.close()
```

Q11. Why is with statement used for file handling?

- Automatically **closes the file** after use.
- Prevents data loss or corruption.
- Cleaner and safer than using `f.close()` manually.

Example:

```
with open("data.txt", "r") as f:
    content = f.read()
    print(content)
# file closed automatically here
```

Q12. What are common errors in file handling and how can we prevent them?

Error	Reason	Prevention
FileNotFoundError	File doesn't exist	Use try-except
PermissionError	No write permission	Check file permissions
Data loss	Using "w" mode wrongly	Use "a" mode or backup file

Example:

```
try:
    f = open("unknown.txt", "r")
    print(f.read())
except FileNotFoundError:
    print("File not found!")
```

Q13. What is the difference between read() and write()?

Function	Purpose	Mode	Example
read()	Reads data from file	"r"	f.read()
write()	Writes data into file	"w"/"a"	f.write("Hello")

Q14. Write a Python program to display only lines containing a particular word (like “Python”).

```
word = "Python"
with open("notes.txt", "r") as f:
    for line in f:
        if word in line:
            print(line.strip())
```

Q15. Write a Python program to display the first N lines of a file.

```
N = 3
with open("sample.txt", "r") as f:
    for i in range(N):
        print(f.readline(), end="")
```